

Instituto Tecnológico de Costa Rica
Centro Académico de Alajuela
IC1803. Taller de Programación



Estudiantes:

Jeremy Fabricio Cubero Aviles – 2026089565

John Chavarría Castillo – 2026106972

Sebastián Rivas Chaves – 2026013223

Profesor:

Ing. Jose Rodolfo Godinez Solano

Fecha de Entrega:

29/05/2026

Resumen Ejecutivo

El presente proyecto tiene como meta implementar un programa capaz de poder resolver laberintos representados por medio de matrices, usando el lenguaje de programación Python. Para dicha resolución se implementaron estrategias de búsqueda basadas en una pila (stack), con funciones auxiliares utilizadas para validar los movimientos, obtener vecinos válidos y marcar el camino encontrado.

El programa va a recibir como entrada un laberinto que contiene espacios libres, un punto inicial, paredes y un punto final. Con esta información, explora sistemáticamente todos los posibles caminos hasta lograr llegar a una ruta que sea válida o verificar si no hay solución existente.

A lo largo del desarrollo se utilizaron estructuras de datos que son fundamentales, como matrices, conjuntos y pilas, permitiendo que se reforzaran al máximo los conocimientos relacionados con la exploración de caminos, organización modular del código. Finalmente, se realizaron diversas pruebas para convalidar el funcionamiento estable del programa en escenarios, ya sean sin solución, con múltiples rutas posibles y con solución.

Introducción

La resolución de laberintos es uno de los problemas más clásicos dentro del área de las ciencias de la computación. Este tipo de problemas hacen posible estudiar técnica de búsqueda, algoritmos para la exploración eficiente de los diferentes espacios de estados y la misma estructura de los datos.

Un laberinto se puede representar por medio de una matriz en la que en esta misma cada celda almacena información sobre espacios transitables, punto de inicio/final y además los obstáculos. El objetivo consta en averiguar una ruta correcta desde la posición inicial hasta la salida (posición final) respetando claramente las restricciones establecidas.

En el presente proyecto se desarrolló como parte del curso Taller de Programación con el fin de aplicar conceptos que se relacionan con matrices, pilas, búsqueda exhaustiva y organización modular del código mediante funciones auxiliares. También, se busca mejorar la capacidad de diseñar soluciones algorítmicas para los problemas de recorrido y exploración.

Marco Teórico

Matrices

Una matriz es una estructura de datos bidimensionales que está compuesta por columnas y filas. Este proyecto, el laberinto está representando por una matriz de caracter que en cada posición almacenara información sobre el tipo de celda que le corresponda.

La simbología que se utilizó es:

- ‘.’ representa un espacio libre.
- ‘S’ representa la posición inicial.
- ‘E’ representa la posición final.
- ‘#’ representa una pared.

Esta representación se utiliza para facilitar el acceso a cualquier posición del laberinto gracias a índices de fila y columna.

Pilas

La pila es una estructura de datos la cual sigue el principio LIFO (Last In, First Out), que significa que el ultimo elemento agregado a su vez, va a eliminarse de primero.

En la implementación utilizada, la pila va a almacenar los estados de exploración pendientes. Con cada estado conteniendo la posicion actual dentro del laberinto y también, el camino que se ha recorrido.

- **append():** inserta los elementos en la pila.
- **pop():** extrae el ultimo elemento agregado.

Conjuntos

Los conjuntos nos permiten almacenar elementos únicos y desarrollar búsquedas de manera eficiente. En este proyecto se utilizaron estos mismo para registrar las posiciones que ya fueron visitadas y recorridos repetidos que podrían generar ciclos innecesarios.

Búsqueda Exhaustiva

La busqueda exhaustiva se refiere en explorar sistematicamente cada una de las posibilidades disponibles hasta descifrar la solución correcta o determinar si esta no tiene solución. Esta metodología de estrategia garantiza que, si hay una ruta de inicio a final, el algoritmo la va a encontrar.

Descripción de la solución

La solución desarrollada es una combinación de matrices, pilas y funciones auxiliares para recorrer el laberinto.

Al principio, el programa va a leer las dimensiones del laberinto y gracias a esto, va a construir una matriz con la información dada por el usuario. Durante esta fase también identifica las posiciones respectivas al punto inicial (S) y final (E).

Seguidamente, se crea una pila que guarda los estados pendientes de exploración, Cada estado contiene la fila actual, la columna actual y la lista de los pasos recorridos actuales. Para evitar visitar muchas veces una misma posición, se debe utilizar un conjunto de nombre ‘visitados’. Esta registra dentro del conjunto cada vez que una posición es explorada.

El algoritmo principal extrae los elementos de la pila y verifica si estos pertenecen al destino. Si la posición actual es la misma que la final, se marca este camino encontrado y muestra la solución. Si no hubiera solución, se calculan los vecinos validos utilizando las funciones auxiliares y nuevos estados son agregados a la pila para continuar con la exploración.

Las funciones auxiliares son:

- **es_posicion_valida():** verific que las posiciones se encuentren dentro de los límites del laberinto, que no sea una pared y que no haya sido visitada.
- **obtener_vecinos():** da las posiciones adyacentes validas.
- **marcar_caminos():** remplaza los espacios libres del recorrido por asteriscos.
- **mostrar_laberinto():** imprime la solución final.
- **resolver_laberinto():** coordina todo el proceso de búsqueda.

Resultado de pruebas

Para poder validar el funcionamiento correcto del programa, es necesario hacer diversas pruebas utilizando las distintas variedades de laberintos, con la meta de evaluar su comportamiento en distintos escenarios.

Prueba 1: Laberinto pequeño

En este caso se pudo verificar el funcionamiento básico del algoritmo en un entorno sencillo.

Resultado: El programa encontró correctamente el camino hacia la meta (salida) desde el punto inicial.

```
C:\Users\Yo\Downloads>python programa.py < laberinto_pequeno.txt
S....
*###.
***.#
****E
```

Prueba 2: Laberinto mediano

Se uso un laberinto de tamaño intermedio con obstáculos.

Resultado: El programa encontró una solución valida, ya mostrando su capacidad en una estructura más compleja.

```
C:\Users\Yo\Downloads>python programa.py < laberintomediano.txt
S*****.
*****#
.#####*.
.#.....#*.
.#.###.#*.
.#.#...#*.
.#.#.###*.
.....#*E
```

Prueba 3: Laberinto grande

En el siguiente caso se aumentó aún más la complejidad en un laberinto de tamaño.

Resultado: El programa encontró una ruta valida en una complejidad mayor.

```
C:\Users\Yo\Downloads>python programa.py < laberintogrande.txt
S*****.#
.#####*.#
.....*.*.#
.#####*.*.#
.#.....#.*.#
.#.#####*.*.#
.#.#...#.#.*.#
.#.#.#.#.#.*.#
.#.#.#.#.#.*.#
.#.#.#.#.#.*.#
.#.....***E
```

Prueba 4: Laberinto con múltiples caminos

En este caso hay no solo una salida, aumento a múltiples rutas posibles.

Resultado: El programa encontró una de las salidas, lo que comprueba que está en funcionamiento la búsqueda exhaustiva.

```
C:\Users\Yo\Downloads>python programa.py < laberintomulti.txt
S****.
*****.
*****.
*****.
*****.
*****E
```

Prueba 5: Laberinto sin solución

En este caso no hay ninguna ruta valida, ósea, no hay forma de lograr a la salida.

Resultado: El programa determina que no hay ruta valida, evitando ciclos repetitivos gracias al conjunto de posiciones visitadas.

```
C:\Users\Yo\Downloads>python programa.py < laberintonosolu.txt
No existe solución
```

Análisis general de los resultados:

- Logra encontrar soluciones cuando estas existen.
- Maneja de manera correcta los diferentes tamaños de laberintos.
- Evade los ciclos y recorridos innecesarios.
- Detecta eficientemente cuando no hay una solución valida (no hay camino hacia la salida).

Conclusiones

1. Se desarrollo un programa funcional que puede resolver laberintos por medio de matrices utilizando Python.
2. Usar una pila permitió implementar eficazmente la exploración sistemática de caminos.
3. Utilizar un conjunto de posiciones visitadas, hizo que no hubiera recorridos repetitivos y la búsqueda mejoro de manera eficiente.
4. Organizar el programa por medio de funciones auxiliares ayudo a la comprensión, reutilización del código y mantenimiento de este.
5. Las pruebas demostraron que el algoritmo logra encontrar soluciones validas y detecta aquellos casos en los que no existe una ruta valida.

Aprendizaje

A lo largo del desarrollo del proyecto se mejoraron o se aprendieron los conocimientos relacionados con pilas, matrices y las técnicas de exploración. También, se obtuvo experiencia en la organización modular de programas, gracias a las funciones auxiliares y ademas de la validación de las soluciones por medio de casos distintos de prueba.

De igual manera, se entendió la importancia de manejar correctamente los estados visitados dentro de un problema de búsqueda, así se evitan ciclos y asegura una ejecución del algoritmo correcta. Finalmente, este proyecto nos dio la oportunidad de desarrollar habilidades para resolver los problemas de programación mediante los métodos utilizados y usar de forma simple los conceptos vistos durante el curso.